HYFEPOUL / FIELD MANUALS

02

Hardware / Technical Manual

Wiring reference, firmware and commissioning

EDITION 1.0 | 27 OCTOBER 2026 | BASELINE d623cce

READERS / Technicians

Measured acceptance required.

Prepared for the October 28 prototype session. Physical acceptance is pending.

IN THIS GUIDE

Architecture and commissioning boundary 02

Component map and signal pins 03

Communication wiring and electrical checks 04

Build and upload the correct firmware 05

Calibrate all five tray load cells 06

Tune travel and verify hopper / water sensing 07

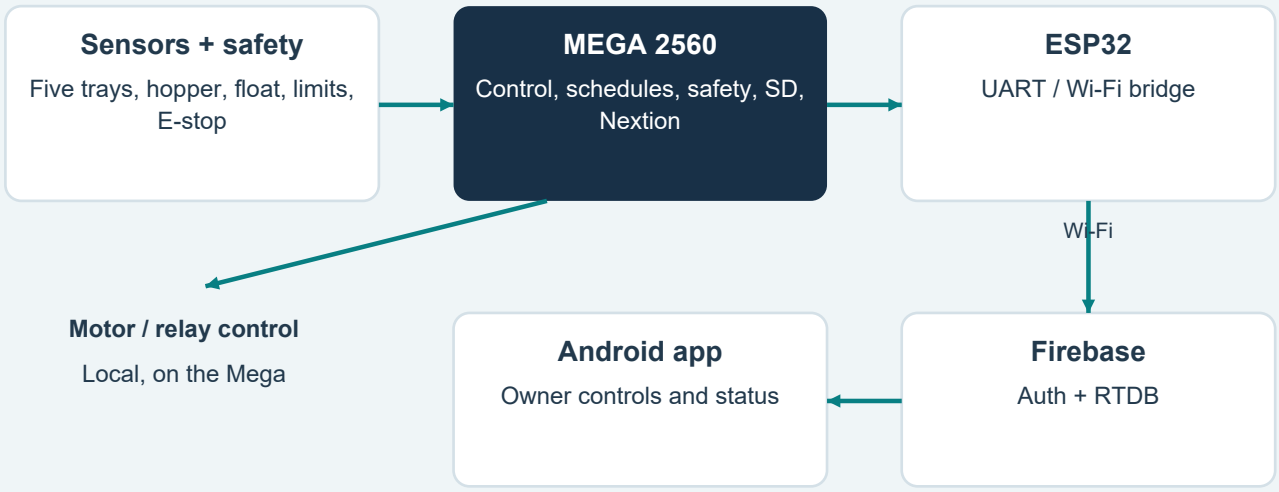
RTC, SD and Nextion commissioning 08

Data flow and command diagnostics 09

Execute the physical acceptance sequence 10

As-built record and maintenance references 11

Architecture and commissioning boundary



Arrows show the data path; commands travel back through the same services.

The Mega 2560 is the primary controller. It reads all sensors, runs schedules and the feed state machine, applies safety checks, controls relays, updates Nextion and logs to SD. The ESP32 bridges UART to Wi-Fi and Firebase. The Android app communicates with Firebase, never directly with the Mega.

The deployed backend path is Firebase Authentication plus Firebase Realtime Database (RTDB). The repository's older Firestore, Cloud Functions and FCM code is not the active implementation. Local app notifications are subject to app and OS execution limits.

Baseline as of 27 October 2026

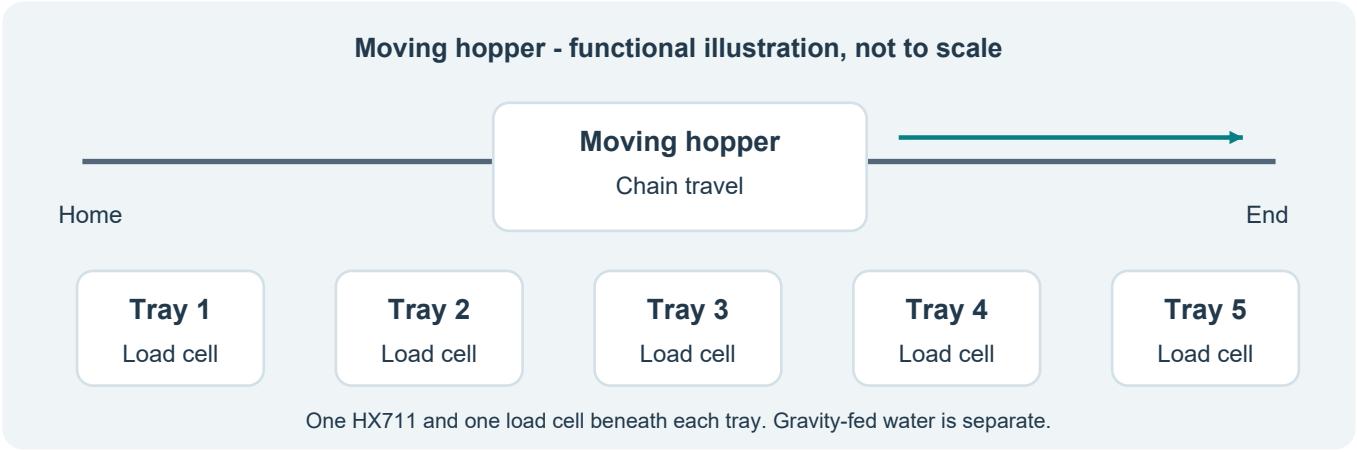
Evidence	Status
Mega and ESP32 compilation	Passed; local commissioning images prepared.
Simulated device, rules and Android emulator checks	Recorded in the repository; do not establish physical safety or accuracy.
Dedicated device Auth account, binding and private PIN	Provisioned; REST sign-in and binding read verified.
Actual Wi-Fi, firmware sign-in, UART, sensors, actuators and phone claim	Physical commissioning still required. Wi-Fi placeholders require a private edit and ESP32 rebuild.

HOLD POINT

Do not energize motors from this document alone. Use the approved power arrangement, powered-off inspection and HIT prerequisites. This is a software pin reference and commissioning guide, not a certified electrical schematic or an as-built drawing.

Source: docs/development-status.md; hardware-test-day-run-sheet.md; firmware baseline d623cce.

Component map and signal pins



One HX711 and one load cell beneath each tray. Gravity-fed water is separate.

Component / signal	Mega pin or port	Software expectation
HX711 DOUT, trays 1 / 2 / 3 / 4 / 5	30 / 40 / 41 / 42 / 43	Five independent tray load cells.
All HX711 SCK	31	Shared clock; boards sampled sequentially.
Hopper ultrasonic TRIG / ECHO	32 / 33	Distance converted to hopper fill estimate.
Reservoir float	34	INPUT_PULLUP; LOW means low water.
Chain forward / reverse relay	36 / 44	HIGH active in code; mutually exclusive.
Dispenser relay	37	HIGH active in code.
E-stop sense / buzzer	38 / 39	E-stop LOW active; buzzer HIGH active.
Home / end limit	45 / 46	INPUT_PULLUP; LOW means switch pressed.
SD CS / SPI	53 / hardware SPI	Mega SPI: 50 MISO, 51 MOSI, 52 SCK.
DS3231 RTC / I2C	20 SDA / 21 SCL	UTC storage, local conversion in firmware.

Confirm each connection against the assembled unit with power off. Do not infer sensor power rails, relay contact ratings, fuse ratings or terminal connections from this table. Component model numbers and the final power circuit must be recorded by the installer.

There is one float switch and gravity-fed water, with no pump. The two chain relays select direction; the dispenser has a separate relay. There are no tray-position sensors: travel between trays is timed and requires physical measurement.

Source: Config.ino; Hydration.ino; Feeding.ino; design-revision-moving-hopper.md.

Communication wiring and electrical checks



Cross TX to RX; verify grounds and signal voltages before power.

Nextion uses the separate Mega Serial2 port: TX 16 / RX 17 at 9600 baud.

Link	Pins	Settings
Mega to ESP32	Mega TX1 18 to ESP32 RX2 16, through suitable 5 V to 3.3 V protection	115200 baud, 8N1, newline ASCII
ESP32 to Mega	ESP32 TX2 17 to Mega RX1 19	Verify logic levels on the actual interface
Mega to Nextion	Mega TX2 16 to Nextion RX; Nextion TX to Mega RX2 17	9600 baud; confirm actual HMI settings
PC to Mega	USB serial	115200 baud, newline for commands

Pins 16 and 17 refer to different headers on each board. Label both cable ends. Verify grounds and level conversion against the electrical drawing before connection.

Powered-off inspection and safe boot

1. Isolate power. Check the pin map, component datasheets and protection for both actuator and logic supplies.
2. Verify level protection prevents 5 V reaching the ESP32 RX. Measure the actual interface voltages.
3. Leave actuator power disconnected. Boot the boards and measure relay output states. Firmware assumes LOW is off; an active-low relay module must not be connected as though HIGH were on.
4. Check limit and E-stop inputs using STATUS while stationary. Pull-ups alone do not prove safe detection of an open cable.
5. Follow the commissioning procedure's separate electrical E-stop check. If its approved method is unavailable, mark it BLOCKED.

ELECTRICAL STOP IS A SEPARATE TEST

The software stop latch and watchdog are not proof that motor power is removed. HIT-06 requires measurement that the physical stop removes actuator power even with a relay-ON condition held by an approved bench method. Never jumper live contacts or reach into the mechanism to simulate a switch.

Source: Config.ino; Safety.ino; docs/hardware-commissioning-procedure.md.

Build and upload the correct firmware

Required source and local configuration

Target	Sketch directory	Board target
Mega 2560	firmware/arduino-mega/HyFePoul_Arduino	arduino:avr:mega
ESP32 bridge	firmware/esp32/HyFePoul_ESP32	esp32:esp32:esp32

Keep each main sketch inside its matching directory; Arduino compiles the sibling .ino files together. Use the project's installed toolchain and libraries from docs/firmware-build-environment.md. ESP32 core 3.3.12 was used for the current build. Keep Core Debug Level at None: verbose HTTP logging can disclose authentication tokens.

Edit the ignored local ESP32 Config.h privately. Confirm Device ID, Firebase project, dedicated device-account credentials and API key, then set the actual 2.4 GHz Wi-Fi SSID/password. Do not publish that file or the resulting ESP32 images, which contain credentials.

Rebuild after changing configuration

Run in PowerShell from the repository root with arduino-cli available. A trailing backtick continues a command onto the next line. Verify the output folders before uploading.

```
arduino-cli board list
arduino-cli compile --fqbn arduino:avr:mega `
--output-dir firmware/build/manuals-local/mega `
firmware/arduino-mega/HyFePoul_Arduino
arduino-cli compile --fqbn esp32:esp32:esp32 `
--output-dir firmware/build/manuals-local/esp32 `
firmware/esp32/HyFePoul_ESP32
```

Connect one board at a time if needed to identify its port. With actuator power isolated, substitute its actual COM port in the commands below. Close serial monitors before upload.

```
arduino-cli upload --fqbn arduino:avr:mega --port COMx `
--input-dir firmware/build/manuals-local/mega
arduino-cli upload --fqbn esp32:esp32:esp32 --port COMy `
--input-dir firmware/build/manuals-local/esp32
```

Upload does not recompile. Do not flash an old output folder after editing Config.h or calibration constants. Do not guess ESP32 raw-image offsets; use the CLI with the matching complete build output.

Verify after upload

Record the commit, board, port, build timestamp and checksum. Observe safe startup and serial messages; confirm all outputs remain off with loads disconnected. Current local builds consumed Mega 49,176 bytes flash / 3,191 bytes RAM and ESP32 1,147,088 bytes flash / 49,688 bytes RAM. These figures are a baseline, not an acceptance limit.

Source: docs/firmware-build-environment.md; firmware READMEs.

Calibrate all five tray load cells

Isolate actuator power for scale work. Fit each tray in its final mounting, remove feed and unintended loads, and allow the assembly to settle. Use known reference masses appropriate to the intended operating range. Record the actual mass, not a guessed package weight.

USB serial procedure

1. Open the Mega serial monitor at **115200 baud**, with newline enabled. Enter **STATUS** and **WEIGH**. Resolve NOT READY or unstable raw readings before calibration.
2. With tray 1 empty and fitted, enter **TARE 1**. Put a known mass on that tray without touching or leaning on it.
3. Enter **CAL 1 <actual grams>**. For a measured 100 g reference only, the command is **CAL 1 100**.
4. Enter **WEIGH** and compare the result to the reference. Remove and replace the mass to check repeatability; repeat with other masses across the intended range.
5. Repeat TARE and CAL separately for trays 2, 3, 4 and 5. Do not copy one tray's factor to the others without measurement.
6. Enter **FACTORS**. Save the reported array into LOAD_CELL_CALIBRATION_FACTORS in Config.ino. Rebuild, upload, reboot, and repeat the known-mass checks.

CAL IS RAM-ONLY

CAL changes the running factor until reboot. FACTORS prints the code line; it does not save it automatically. The checked-in -7050.0 values are placeholders and must not be treated as final calibration.

Calibration worksheet

Tray	Reference g	Measured g	Factor	Reboot check / initials
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____

Record repeatability and measurement error in the commissioning log. The team must define an acceptable error against the actual feeding requirement before accepting the calibration; this manual does not invent a precision specification.

The normal feed cycle tares each tray before dispensing. A displayed tray value therefore measures change since that tare, not total historical inventory. CAL, TARE and WEIGH take blocking samples; perform them only with machinery stationary and isolated.

Source: SerialConsole.ino; Config.ino; Feeding.ino; HIT-08.

Tune travel and verify hopper / water sensing

Motion calibration - supervised technician only

After wiring, relay, E-stop and limit-switch prerequisites pass, use an unloaded hopper for initial motion checks. Keep people clear of the path and maintain an observer at the physical stop. Begin with the shortest useful movement; do not send a full cycle using unmeasured travel values.

Console command	Action / range
JOG F <ms> / JOG R <ms>	Forward or reverse movement; 100-5000 ms.
HOME	Reverse toward the home switch; 30 s configured timeout.
DISP <ms>	Dispenser run; 100-5000 ms. Use only when its outlet is safely controlled.
STOP	Stop a bench move. It is not a substitute for the physical E-stop.
FEED <grams>	Full five-tray cycle; use only after calibration and safety acceptance.
RESET	Clear the stop latch only when the physical E-stop is released and the area is safe.

Measure incremental travel from home to tray 1, then tray 1 to 2, 2 to 3, 3 to 4 and 4 to 5. Save the five measured durations in TRAY_TRAVEL_MS and rebuild. Record end and home travel and justify timeout margins from measured behavior. Test with representative load after unloaded validation; timed positioning can change with load, friction and supply conditions.

Current constant	Baseline only - verify physically
Each tray travel	3000 ms placeholder
Tray settle / direction dead time	500 ms / 300 ms
Dispense / end / home timeout	30 s / 15 s / 30 s
Tare sampling	5 samples within 3 s

Hopper and reservoir

Measure the ultrasonic distance at the actual empty and full hopper levels. Replace HOPPER_EMPTY_DISTANCE_CM (40) and HOPPER_FULL_DISTANCE_CM (5) with measured values, then rebuild. Check intermediate levels and no-echo behavior. Do not fill beyond the mechanically approved level just to obtain a sensor endpoint.

Move the float through actual safe low and filled states. LOW on pin 34 means low reservoir; the firmware debounces for 2 s. Verify installation orientation and wiring rather than silently accepting reversed indications. This sensor does not confirm that water is flowing to every drinker.

Source: SerialConsole.ino; Config.ino; FeedLevel.ino; Hydration.ino; HIT-07 to HIT-10.

RTC, SD and Nextion commissioning

Clock and persistent schedules

The DS3231 stores UTC. The Mega applies UTC+8 to schedule matching and the Nextion clock. The ESP32 obtains network time and sends UTC to the Mega; the Mega falls back to the RTC when network time is no longer fresh. Check Network time, Backup clock and Clock not set states in the app.

Set and verify a near-future test schedule with actuator power safely controlled, then power-cycle and confirm EEPROM persistence. Verify local behavior during Wi-Fi loss. Fresh defaults are 07:00, 12:00 and 18:00 at 50 g per tray, enabled; a cloud restore or EEPROM load can replace these. An invalid clock is a commissioning failure for unattended scheduling.

SD logging

The Mega writes periodic rows about every 30 seconds and logs events to monthly **HFyyyymm.CSV** files. Timestamps are UTC. Retention is 24 months in the current implementation; archive required evidence before retention cleanup. With power isolated, remove and read the card and verify both regular rows and deliberate test events. Restore the card before the next run and confirm initialization succeeds.

Nextion display and touch IDs

Home field	Firmware value
t2	Total grams in five trays; an older HMI may label this Hopper Load.
t3	Water state.
t4	Cycle/status text.
t5	Local HH:MM:SS.
t0 / t1 / t6 / t7	Static captions / grams unit, not data fields.

The current Home page has no hopper-percentage or network-status field. The project is Nextion/ChickenFeederV1 (1).hmi; use the Nextion Editor and actual display model to generate a matching TFT file. A successful HMI compile does not prove the display transfer or touch wiring.

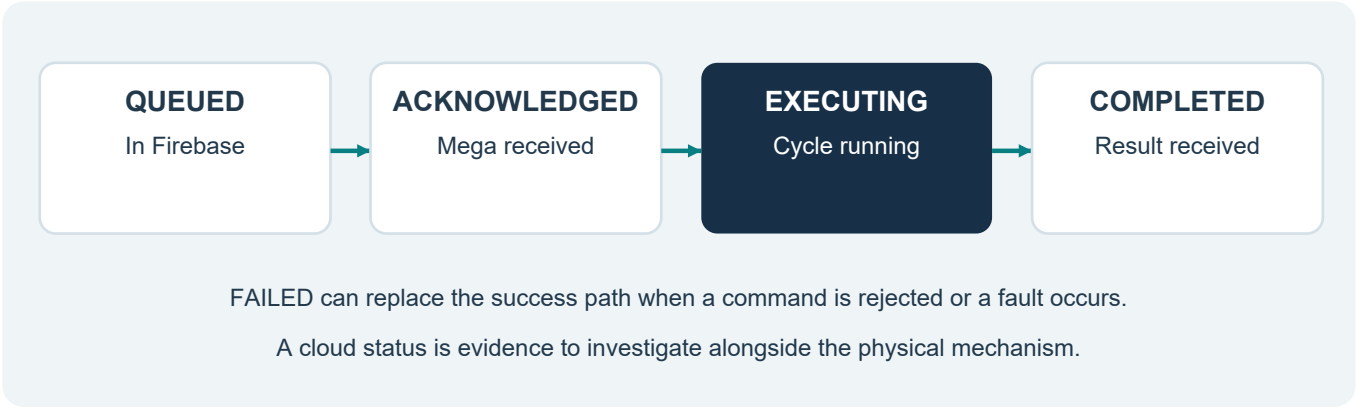
With actuator loads disconnected, ensure Send Component ID is enabled for the intended controls. Press Stop and Start separately and record the USB messages **Nextion touch: page P id C**. Verify both press and release behavior. Only then replace the disabled 255 constants for the Home page and the two buttons in Nextion.ino, rebuild and retest the stop/reset behavior.

UNFINISHED TOUCH FUNCTIONS

b3 Stop and b4 Start are disabled until verified. b0 Set Feed Schedule, b1 Log Book and b2 Set Feed Weight pages are not implemented in Mega handlers. Do not mark them functional from the HMI appearance alone.

Source: TimeManager.ino; ScheduleStorage.ino; SDLogging.ino; Nextion.ino; Nextion_IDs.txt.

Data flow and command diagnostics



FAILED can replace the success path when a command is rejected or a fault occurs.

A cloud status is evidence to investigate alongside the physical mechanism.

UART contract

The link uses newline-terminated ASCII with pipe-delimited fields at 115200 baud. ESP32-to-Mega command lines are capped at 63 characters before newline and paced at least 50 ms apart. Oversized partial lines are discarded. Use docs/mega-esp32-protocol.md and the current source for full message definitions.

```
DATA|tray1|tray2|tray3|tray4|tray5|targetPerTray|
hopper%|waterLow|feedingActive|cycleStep|
currentTray|emergencyStop|timeSource
```

The above is a wrapped field diagram; an actual DATA packet is one line. Cycle states include IDLE, HOMING, TO_TRAY, TARING, DISPENSING, TO_END and RETURNING. The Mega sends system data about every 10 seconds. Cloud history is normally every 5 minutes plus feed start/end; the SD interval is 30 seconds.

Diagnostic	Current behavior
No Mega ACK	UART_TIMEOUT after 3 s waiting for acknowledgement.
No execution result	EXECUTION_TIMEOUT after 300 s; inspect the physical state before retry.
Old command	Commands older than 5 minutes are rejected as stale.
Mega silent	ESP32 marks controller unavailable after 30 s.
ESP32 silent	Mega link status expires after 15 s.
Heartbeat stale	App marks device offline after 90 s; checked periodically.

Firebase writes wait for valid network time. Schedule restores occur after reconnect and Mega restart, with retries after failures. The ON DEVICE marker is a device acknowledgement, not merely a saved cloud record. Missing cloud slots can disable local slots during a successful restore; failed reads retain the existing EEPROM schedule.

Events have bounded in-memory buffering and no UART acknowledgement. Power loss can lose unsent events. Treat the SD files and commissioning observations as important additional evidence; cloud alerts are not a complete event sequence.

Source: HyFePoul_ESP32.ino; ESP32Communication.ino; docs/mega-esp32-protocol.md.

Execute the physical acceptance sequence

Use the detailed HIT matrix and run sheet. This order is a summary, not a replacement for their expected results. Record PASS only from observations on the assembled system; otherwise record FAIL or BLOCKED with the reason.

Gate	Checks	Required evidence before advancing
1. Wiring	HIT-01	Measured power/ground checks, pin map, level protection, actuator isolation.
2. Safe boot	HIT-02	Relays off, Mega startup, RTC, SD and EEPROM behavior.
3. Communication	HIT-03, HIT-04	ESP32 hardware authentication, matching UID, TLS, state writes, UART and reconnect.
4. App path	HIT-05; claim part of HIT-15	Actual production phone claim, status, schedule delivery and safe command behavior with motors disconnected.
5. Actuator safety	HIT-06, HIT-07, HIT-11	Electrical and software E-stop, limits, correct direction and relay interlock.
6. Sensors / display	HIT-08 to HIT-10, HIT-12	Five scale calibrations, hopper endpoints, float polarity, Nextion, RTC and SD checks.
7. Recovery	HIT-13, HIT-14	Stored schedule after reboot, network loss, controller restart and safe sensor-fault behavior.
8. Complete feed	HIT-16; final HIT-15	Five trays, measured quantities, return home, empty-hopper and blocked-outlet checks, matching phone data.

Do not connect actuator power until HIT-01 through HIT-05 pass. Full feeding comes after safety and sensor prerequisites. Initial motion tuning is unloaded; a loaded feed demonstration is last.

Record every revision

For each failed check, note the expected result, observation, firmware/build, physical wiring revision, measurements and evidence location. Correct the issue and repeat the failed check and dependent checks. Do not overwrite a failure with an unexplained PASS.

Mega watchdog testing must be a controlled idle bench check; WDTEST deliberately stalls the controller. Verify bootloader/watchdog recovery on the actual board. Do not use it during motion. The ESP32 watchdog is also not a substitute for electrical actuator protection.

RELEASE CONDITION

Compiled firmware is ready for supervised commissioning, not automatically for unattended use. Client handover also requires the actual client account and a controlled ownership transfer if a test owner was used.

Source: docs/hardware-test-day-run-sheet.md; hardware-integration-test-plan.md; hardware-commissioning-procedure.md.

As-built record and maintenance references

Complete at the bench

Record	Measured / approved value
Device ID / unit location	_____
Mega / ESP32 board identifiers	_____
Firmware commit / build timestamps	_____
Wiring drawing revision / relay polarity	_____
Load-cell factors 1 through 5	_____
Incremental travel times 1 through 5	_____
Empty / full hopper distances	_____
Float / limit / E-stop polarity verified	_____
Nextion project / page and button IDs	_____
Electrical E-stop test evidence	_____
Test date / technician / reviewer	_____

Keep the five-tray calibration worksheet, error measurements, HIT matrix and raw test evidence with this record. Keep credentials and the setup PIN in a separate private handover, not this record or Git.

Change control

Recheck affected safety and calibration tests after changing a relay module, supply, load cell, tray mounting, chain mechanism, motor, limit switch, timing constant or stop input. Rebuild and record new checksums after any firmware configuration change. Recheck account binding after any device-account change.

Source index

- docs/development-status.md and docs/remaining-work-roadmap.md: current readiness and remaining work.
- docs/hardware-integration-test-plan.md, docs/hardware-commissioning-procedure.md and docs/hardware-commissioning-log.md: detailed acceptance and evidence.
- docs/firmware-build-environment.md: installed cores, libraries, local artifacts and checksums.
- docs/mega-esp32-protocol.md and docs/database-schema.md: communication and data model.
- firmware/arduino-mega/HyFePoul_Arduino/: Config, Feeding, Safety, SerialConsole, TimeManager, ScheduleStorage, SDLogging and Nextion modules.
- firmware/esp32/HyFePoul_ESP32/HyFePoul_ESP32.ino: bridge, authentication, commands and cloud writes.

Edition 1.0, 27 October 2026. Source baseline d623cce. The diagrams are functional illustrations, not scale drawings. Update this edition from measured commissioning results before final client release.